

1. 解耦部门编码与主键：引入内部唯一标识（如自增 ID）作为部门主键，部门编码改为业务属性，支持编码变更而不影响关联关系。

设计部门版本管理：建立部门历史表，记录部门变更前后的编码、层级、时间戳等，业务数据通过“部门 ID+生效时间”追踪归属变化。

抽离部门关系规则：将友好/互斥关系从硬编码改为数据驱动，通过配置表存储部门关系，并在权限控制中动态解析。

重构业务逻辑判断：将依赖具体编码的流程改为基于部门属性（类型、层级、标签）或策略模式，避免编码硬编码。

设计适配层：为旧数据提供编码映射转换服务，确保数据升级时不丢失且可回溯。

预留扩展接口：支持未来组织结构调整时，通过配置而非修改代码实现。

2. 软件工程方面：

采用重构与设计模式（如策略模式处理部门关系、适配器模式处理编码映射）。

引入数据库版本管理（如 Flyway）和数据迁移脚本，保证结构变更可回滚。

编写单元测试和集成测试，重点覆盖部门关系、历史追踪、编码转换逻辑。

进行设计评审和代码走查，确保解耦和扩展性。

项目管理方面：

制定质量计划，明确关键质量目标（数据不丢失、历史可追溯、编码可变更）。

采用迭代开发与原型验证，先解决核心风险（编码解耦、关系配置化）。

建立测试准入、准出标准，对历史数据迁移进行全量验证。

加强配置管理和变更控制，防止范围蔓延影响设计质量。

定期开展技术风险评估，重点关注遗留系统硬编码改造的回归风险。